

SQL PROJECT ON PIZZA SALES

by Israt Jahan Chandmoni

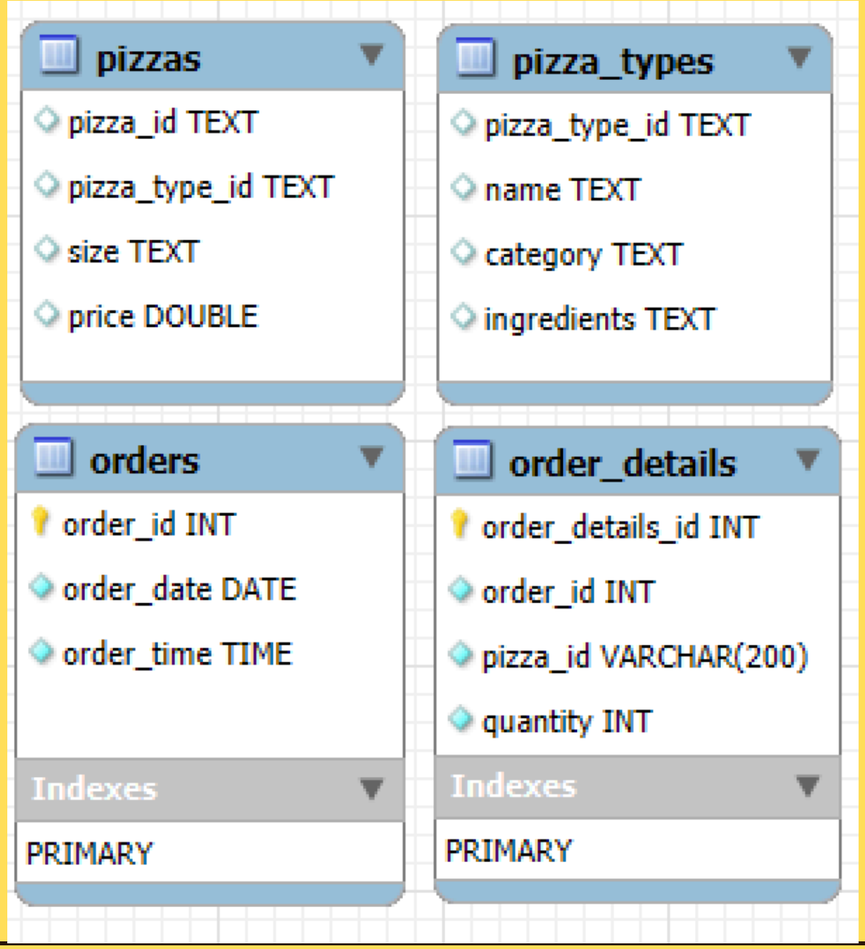


HELLO!!!

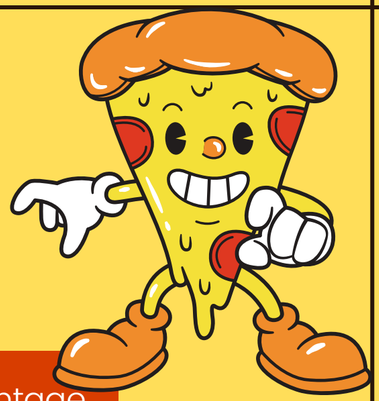
I have created this pizza sales project using SQL to analyze data, including total sales, top-selling pizzas, and order patterns. The project showcases basic and advanced SQL queries for business insights.



LET'S INTRODUCE THE DATASET



PROJECT QUESTIONS:



01 Retrieve the total number of orders placed.

02 Calculate the total revenue generated from pizza sales.

03 Identify the highest-priced pizza.

04 Identify the most common pizza size ordered.

05 List the top 5 most ordered pizza types along with their quantities.

06 Join the necessary tables to find the total quantity of each pizza category ordered.

07 Determine the distribution of orders by hour of the day.

08 Join relevant tables to find the category-wise distribution of pizzas.

09 Group the orders by date and calculate the average number of pizzas ordered per day.

10 Determine the top 3 most ordered pizza types based on revenue.

11 Calculate the percentage contribution of each pizza type to total revenue.

12 Analyze the cumulative revenue generated over time.

13 Determine the top 3 most ordered pizza types based on revenue.

TO LOAD THE FILES FROM CSV TO SQL, I FIRST CREATED THE TABLE AND THEN LOADED THE FILE USING A QUERY, WHICH SAVED A LOT OF TIME.



1 QUERY GIVEN FOR EXAMPLE

```sql
create table orders(
order_id int not null primary key,
order_date date not null,
order_time time not null
);
```

```sql
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/orders.csv'
INTO TABLE orders
FIELDS TERMINATED BY ','
IGNORE 1 LINES;
```

01. Retrieve the total number of orders placed.

```sql
-- Retrieve the total number of orders placed.
SELECT
COUNT(*) AS total_orders
FROM orders;
```

Result Grid

	total_orders
▶	21350



02. CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES.

```sql
-- calculate the total revenue generated from pizza sales
SELECT round(SUM(p.price * od.quantity),2) AS total_revenue
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id;
```

Result Grid

total_revenue
817860.05



03. IDENTIFY THE HIGHEST-PRICED PIZZA.

```sql
-- Identify the highest prices pizza.
SELECT pt.name,  p.price
FROM pizzas p
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
ORDER BY p.price DESC
LIMIT 1;
```

Result Grid | Filter Rows

name	price
The Greek Pizza	35.95

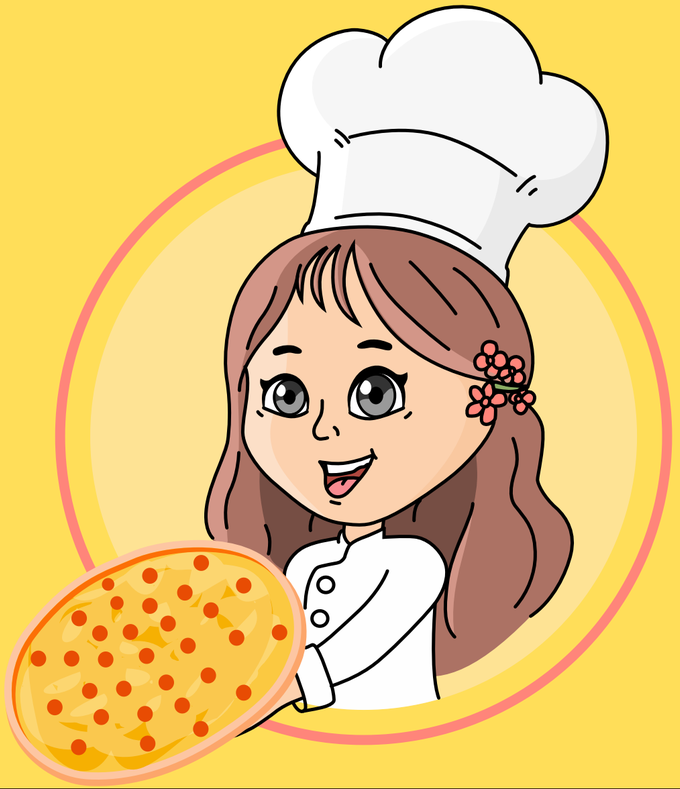


04. IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED.

```sql
-- Identify the most common pizza size ordered.
SELECT p.size, COUNT(od.pizza_id) AS order_count
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
GROUP BY p.size
ORDER BY order_count DESC;
```

Result Grid | Filte

size	order_count
L	18526
M	15385
S	14137
XL	544
XXL	28

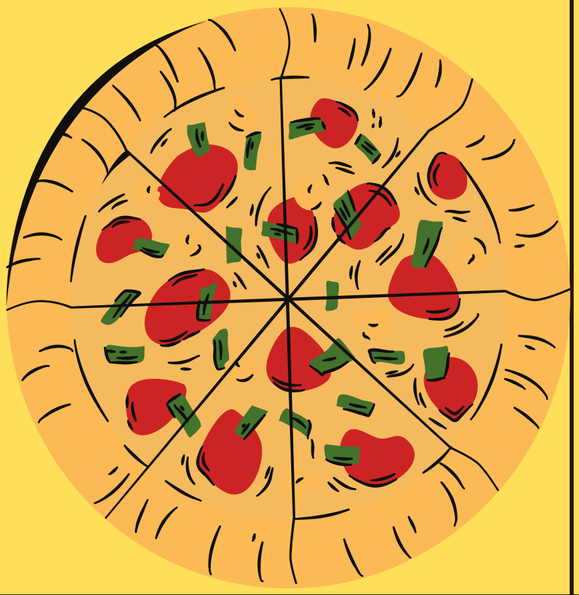


05. LIST THE TOP 5 MOST ORDERED PIZZA TYPES ALONG WITH THEIR QUANTITIES.

```sql
-- List the top 5 most ordered pizza types along with their quantities.
SELECT pt.name AS pizza_type, SUM(od.quantity) AS total_quantity
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_quantity DESC
LIMIT 5;
```

Result Grid | Filter Rows:

pizza_type	total_quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

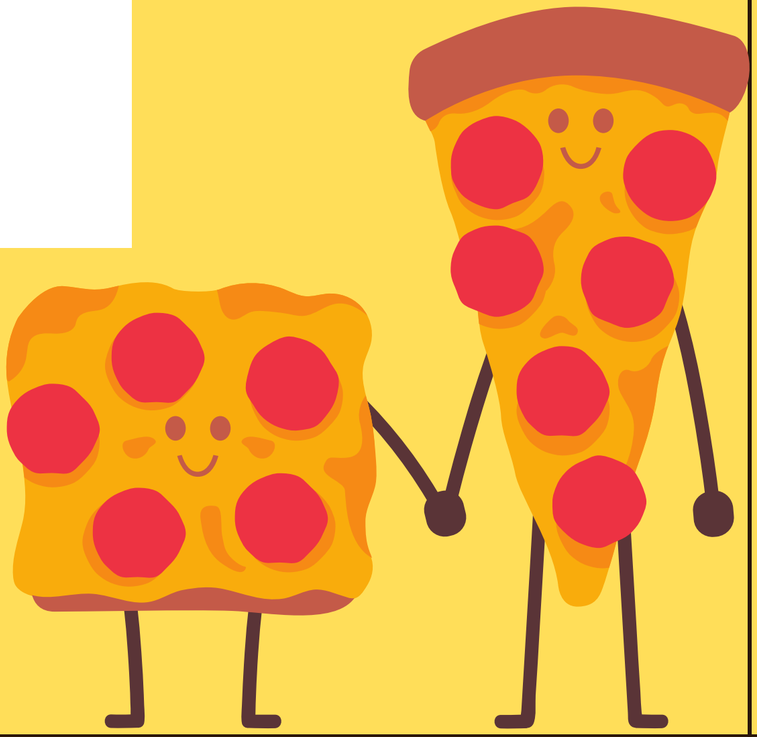


06. JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED.

```sql
-- Join the necessary tables to find the total quantity of each pizza category ordered.
SELECT pt.category AS pizza_category, SUM(od.quantity) AS total_quantity
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY total_quantity DESC;
```

Result Grid | Filter Rows:

pizza_category	total_quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050



07. DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY.

```sql
-- Determine the distribution of orders by hour of the day.
SELECT EXTRACT(HOUR FROM order_time) AS hour_of_day, COUNT(*) AS total_orders
FROM orders
GROUP BY EXTRACT(HOUR FROM order_time)
ORDER BY hour_of_day;
```



Result Grid | Filter Rows:

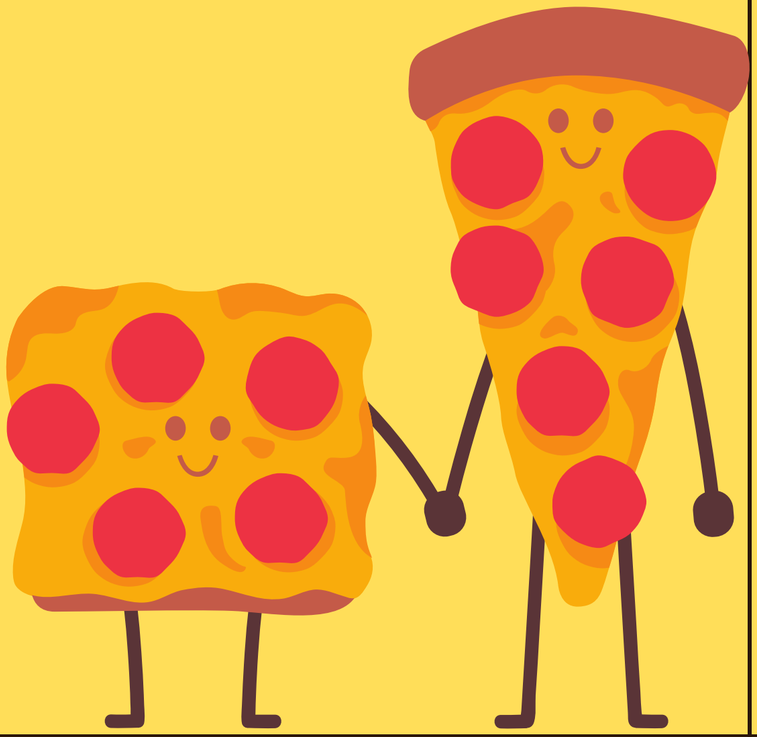
hour_of_day	total_orders
9	1
10	8
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28

08. JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS.

```sql
-- Join relevant tables to find the category-wise distribution of pizzas.
SELECT pt.category AS pizza_category, COUNT(p.pizza_id) AS number_of_pizzas
FROM pizzas p
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY number_of_pizzas DESC;
```

Result Grid | Filter Rows:

pizza_category	number_of_pizzas
Veggie	27
Classic	26
Supreme	25
Chicken	18

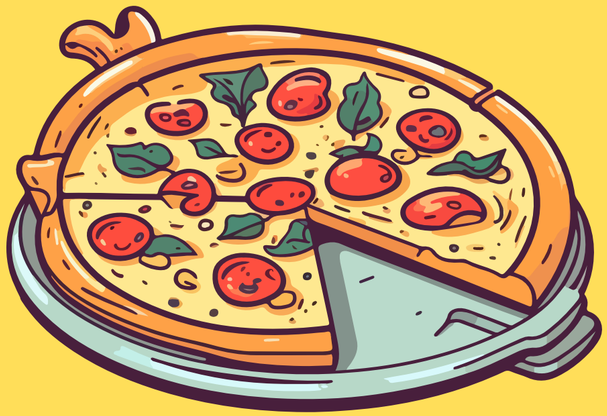


09. GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY.

```sql
-- Group the orders by date and calculate the average number of pizzas ordered per day.
SELECT AVG(daily_pizza_count) AS average_pizzas_per_day
FROM (
    SELECT o.order_date, COUNT(od.pizza_id) AS daily_pizza_count
    FROM orders o
    JOIN order_details od ON o.order_id = od.order_id
    GROUP BY o.order_date
) AS DailyOrderCounts;
```

Result Grid | Filter R

	average_pizzas_per_day
▶	135.8101

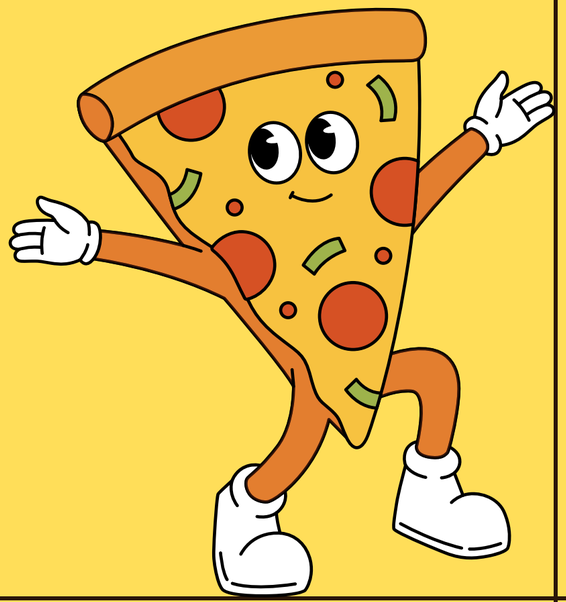


10. DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE.

```sql
-- Determine the top 3 most ordered pizza types based on revenue.
SELECT pt.name AS pizza_type, SUM(p.price * od.quantity) AS total_revenue
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows:

pizza_type	total_revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5



11. Calculate the percentage contribution of each pizza type to total revenue.

```sql
-- Calculate the percentage contribution of each pizza type to total revenue.
SELECT
    pt.name AS pizza_type,
    SUM(p.price * od.quantity) AS total_revenue,
    (SUM(p.price * od.quantity) / (SELECT SUM(p.price * od.quantity) FROM order_details od
                                   JOIN pizzas p ON od.pizza_id = p.pizza_id)) * 100 AS percentage_contribution
FROM
    order_details od
JOIN
    pizzas p ON od.pizza_id = p.pizza_id
JOIN
    pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY
    pt.name
ORDER BY
    percentage_contribution DESC;
```



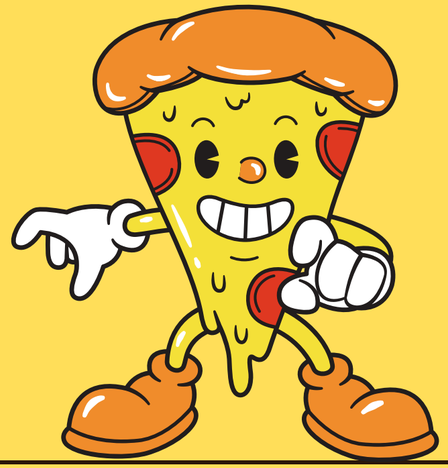
Result Grid | Filter Rows: | Export: | Wrap Cell Co

pizza_type	total_revenue	percentage_contribution
The Thai Chicken Pizza	43434.25	5.310719113863108
The Barbecue Chicken Pizza	42768	5.2292565213327595
The California Chicken Pizza	41409.5	5.063152308270878
The Classic Deluxe Pizza	38180.5	4.668341484585331
The Spicy Italian Pizza	34831.25	4.258827656394306
The Southwest Chicken Pizza	34705.75	4.243482732773205
The Italian Supreme Pizza	33476.75	4.093212524563375
The Hawaiian Pizza	32273.25	3.9460602092008625
The Four Cheese Pizza	32265.70000000065	3.945137068377521
The Sicilian Pizza	30940.5	3.7831044565632306
The Pepperoni Pizza	30161.75	3.687886454412373
The Greek Pizza	28454.100000000013	3.4790915634038195
The Mexicana Pizza	26780.75	3.2744905439506713
The Five Cheese Pizza	26066.5	3.187158976648905
The Pepper Salami Pizza	25529	3.121438686239806
The Italian Capocollo Pizza	25094	3.068251102373837
The Vegetables + Vegetable...	24374.75	2.9803081835333822
The Prosciutto and Arugula ...	24193.25	2.9581161226789607
The Napolitana Pizza	24087	2.945124902481813
The Spinach and Feta Pizza	23271.25	2.8453828989446546
The Big Meat Pizza	22968	2.8083044281231486
The Pepperoni, Mushroom, ...	18834.5	2.302900111088708
The Chicken Alfredo Pizza	16900.25	2.0663987683467537

12. Analyze the cumulative revenue generated over time.

```sql
-- Analyze the cumulative revenue generated over time.
SELECT
    o.order_date,
    SUM(p.price * od.quantity) AS daily_revenue,
    SUM(SUM(p.price * od.quantity)) OVER (ORDER BY o.order_date) AS cumulative_revenue
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN orders o ON od.order_id = o.order_id
GROUP BY o.order_date
ORDER BY o.order_date;
```



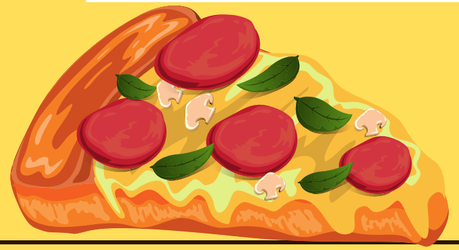


Result Grid | Filter Rows: | Export:

order_date	daily_revenue	cumulative_revenue
2015-01-01	2713.8500000000004	2713.8500000000004
2015-01-02	2731.8999999999996	5445.75
2015-01-03	2662.3999999999996	8108.15
2015-01-04	1755.4500000000003	9863.6
2015-01-05	2065.95	11929.55
2015-01-06	2428.95	14358.5
2015-01-07	2202.2000000000003	16560.7
2015-01-08	2838.3499999999995	19399.05
2015-01-09	2127.3500000000004	21526.4
2015-01-10	2463.95	23990.350000000002

13. DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY.

```sql
-- Determine the top 3 most ordered pizza types
-- based on revenue for each pizza category.
SELECT
    pt.category,
    pt.name AS pizza_type,
    SUM(p.price * od.quantity) AS total_revenue
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category, pt.name
ORDER BY pt.category, total_revenue DESC;
```



Result Grid | Filter Rows: | Export:

category	pizza_type	total_revenue
Chicken	The Thai Chicken Pizza	43434.25
Chicken	The Barbecue Chicken Pizza	42768
Chicken	The California Chicken Pizza	41409.5
Chicken	The Southwest Chicken Pizza	34705.75
Chicken	The Chicken Alfredo Pizza	16900.25
Chicken	The Chicken Pesto Pizza	16701.75
Classic	The Classic Deluxe Pizza	38180.5
Classic	The Hawaiian Pizza	32273.25
Classic	The Pepperoni Pizza	30161.75
Classic	The Greek Pizza	28454.100000000013
Classic	The Italian Capocollo Pizza	25094
Classic	The Napolitana Pizza	24087
Classic	The Big Meat Pizza	22968
Classic	The Pepperoni, Mushroom, ...	18834.5
Supreme	The Spicy Italian Pizza	34831.25
Supreme	The Italian Supreme Pizza	33476.75
Supreme	The Sicilian Pizza	30940.5
Supreme	The Pepper Salami Pizza	25529

THANK YOU!

